



Refactoring Task

[Visit our website](#)

Introduction

Refactoring is important to ensure overall quality code. Refactoring is changing the structure of a program without changing its functionality. When refactoring your code, you take code that is working, but make changes to ensure that your code adheres accurately to best practice guidelines. This lesson introduces you to the concept of refactoring, as well as the variety of tools provided by Eclipse IDE that allow you to refactor your code quickly and easily.

Refactoring doesn't change the behaviour of the code and it doesn't remove errors. Rather, it has to do with restructuring code to make it more readable, maintainable, and testable. Refactoring takes code that works and simply improves it.

Software developer and author Martin Fowler defines refactoring as “a change made to the internal structure of the software to make it easier to understand and cheaper to modify without changing its observable behaviour” (Fowler, 1999). The word “refactoring” in modern programming grew out of software engineer Larry Constantine's original use of the word “factoring” in structured programming, which referred to decomposing a program into its constituent parts as much as possible.

Before we start refactoring code, note this important point: use version control! When you are changing code, you always run the risk of 'breaking' it. Therefore, always make sure that you use version control so that if a change doesn't have the desired effect, you can go back to a previously working version of your code. If you need a refresher on using Git for version control, refer back to the lessons that cover this topic.

Refactoring basics

Let's begin by looking at a few refactoring basics. This includes doing some of the following:

- **Use meaningful names:** Throughout your code, make sure that all variables, functions, modules, classes, etc., are given meaningful, descriptive names. Consider the following example of variable naming:

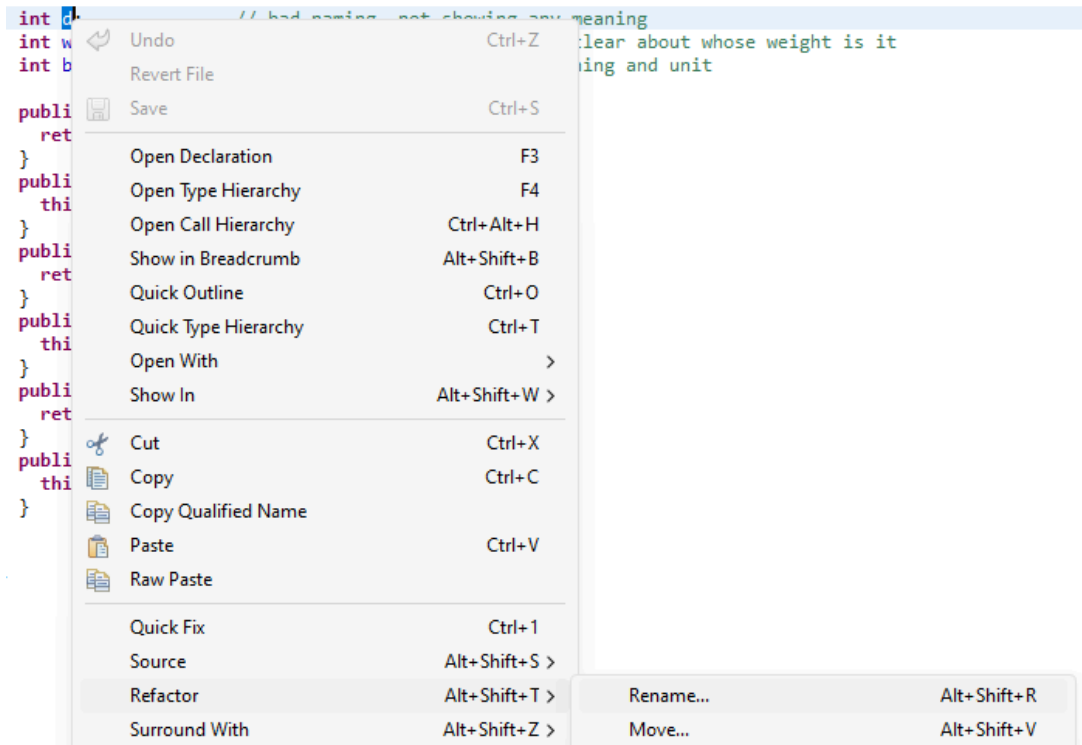
```
int d; // Unclear and poorly named variable  
  
int weight; // Improved, but lacks specificity about the type of weight  
  
int bodyWeightInKg; // Clearly named variable indicating body weight in kilograms
```

You can rename an element and all of its appearances in Eclipse IDE. As an example, you can rename the variable `d` to a meaning name `ageInDays` in the sample code above by:

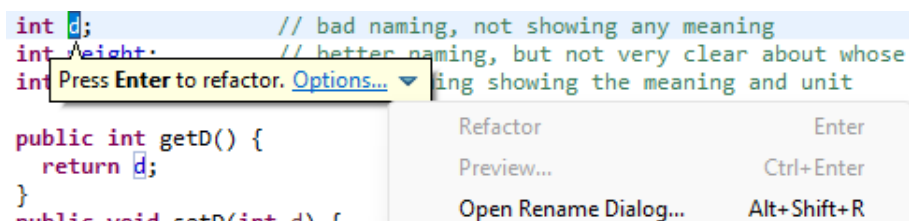
1. Clicking on the Java element in the Explorer view, or selecting it in a Java source file.

```
int d;
```

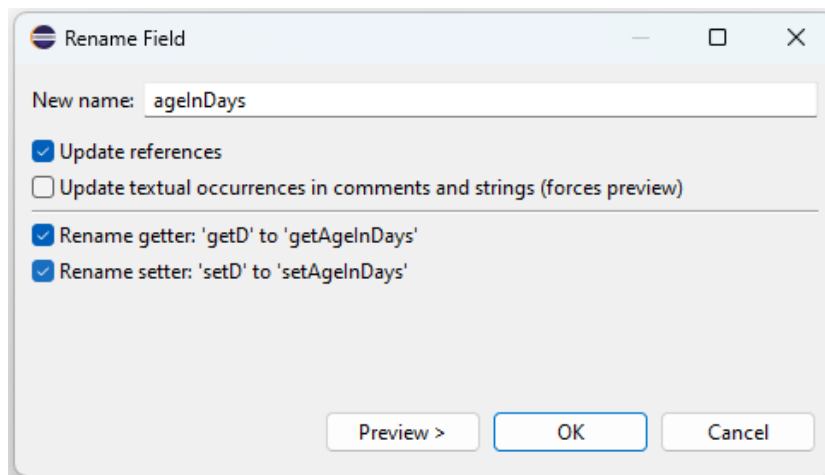
2. Now, right-click and select **Refactor** → **Rename** from the menu.



3. There are two options to rename the variable here. You can enter the new variable name in place, or you can select **Options** → **Open Rename Dialog** to have a finer control over the renaming. Let's select the **Open Rename Dialog** as an illustration.



4. **Rename Field** dialogue box will pop up. Enter the new name for the variable `ageInDays`.



5. In the **Rename Field** dialogue, you can also select **Rename getter** and **Rename setter** to update the names of these methods to reflect the new field.
6. You can click **Preview** to see the changes that Eclipse IDE proposes to make after you have specified everything necessary to perform the refactoring. This lets you reject or approve each change in each affected file, individually.
7. Otherwise, you can just click **OK** if you have confidence in Eclipse IDE's ability to perform the change correctly. The variable is renamed as shown below:

```
int ageInDays;  
int weight; // Improved, but lacks specificity about the type of weight  
int bodyWeightInKg; // Clearly named variable indicating body weight in kilograms
```

- **Make sure that your code is broken down into modules:** (functions, classes, methods, etc.). When refactoring, it's important to change the structure of the code so that chunks of code are extracted to appropriate units, such as functions or modules. There are many benefits to using modular code. Having modular code helps avoid duplicating code. When you have duplicated code, whenever you make changes in one place, you have to make parallel changes in another place. This sets you up to make parallel modifications. It also violates the "DRY" principle: Don't Repeat Yourself.

Using modular code can also make your code easier to read, test, and debug.

In Eclipse IDE, it's easy to extract a unit of code into a method. Consider the following code:

```
// Get a string of the number  
String temp = "";
```

```

for (int j = 0; j < MAX_DIGIT_LENGTH; j++) {
    temp = temp + String.valueOf(command.charAt(i));

    if (!Character.isDigit(command.charAt(i)) || i >=
command.length()) {

        Break;
    }

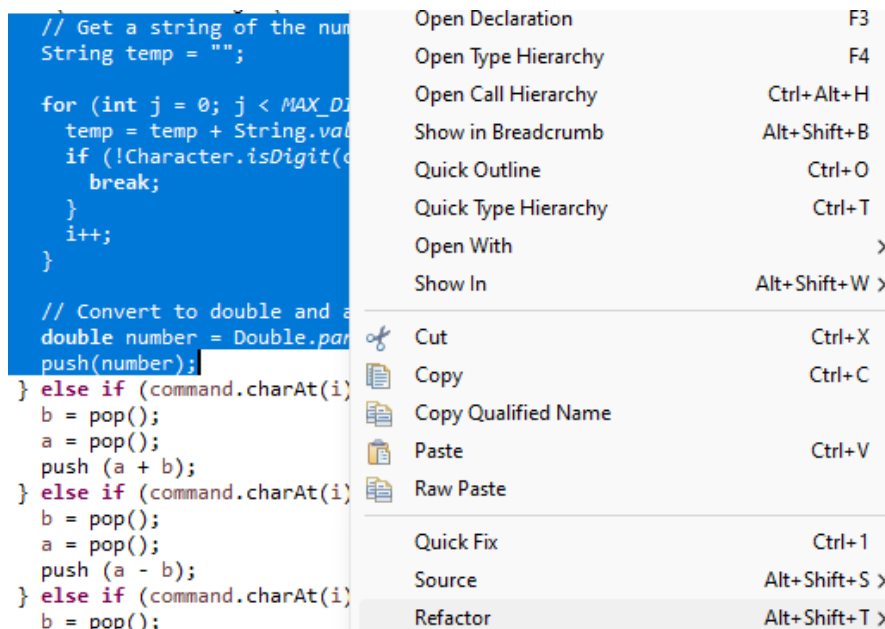
    I++;
}

// Convert to double and add to the stack
double number = Double.parseDouble(temp);
push(number);

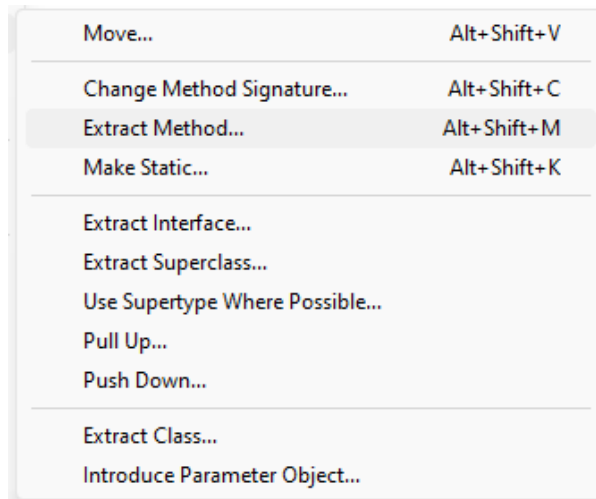
```

You can extract it into a method in Eclipse IDE by:

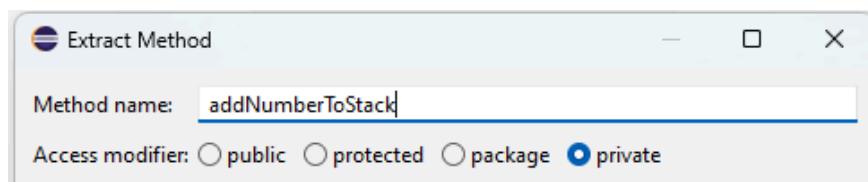
1. Highlighting the section of code.
2. Right-click and select **Refactor** from the menu.



3. A menu with suggested refactor actions should pop up. Select **Extract Method**.



4. You'll be prompted for a method name. A new method that contains the code you extracted will then be created.



5. The call to the method will also be created in the appropriate place.

```
for (int i = 0; i < command.length(); i++) {  
    if (Character.isDigit(command.charAt(i))) {  
        i = addNumberToStack(command, i);  
    } else if (command.charAt(i) == '+') {  
        b = pop();  
    }  
}
```

You can use a similar method to extract local variables or constants in Eclipse IDE. For example, consider the code below:

```
/**  
 * Continuously prompts the user for an RPN expression or "quit" to exit,  
 * processes the expression, and prints the result.  
 */  
public static void main(String[] args) {  
    while (true) {  
        Scanner in = new Scanner(System.in);  
        System.out.println("Enter RPN expression or \"quit\".");
```

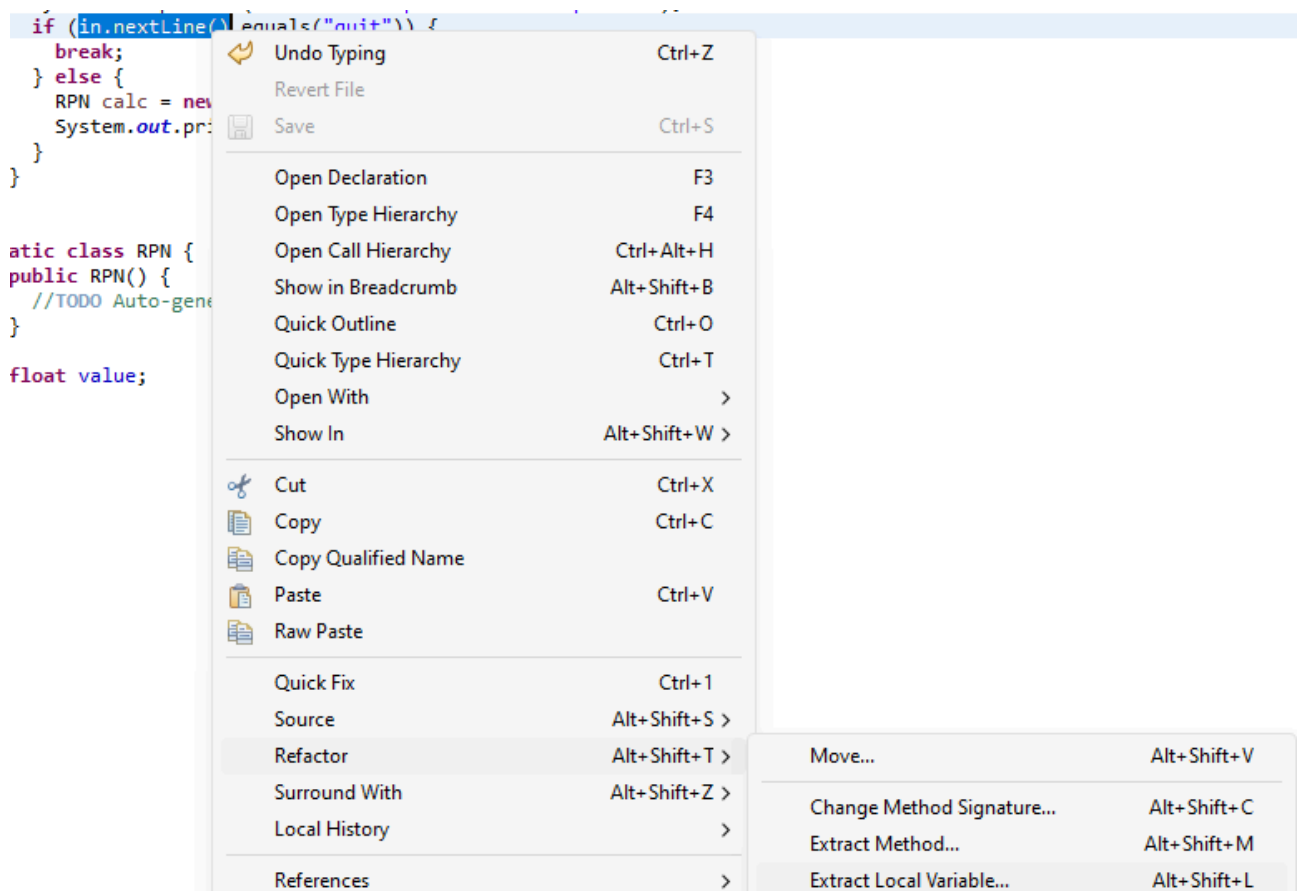
```

        if (in.nextLine().equals("quit")) {
            Break;
        } else {
            RPN calc = new RPN();
            System.out.printf("Answer is %f\n", calc.value);
        }
    }
}

```

You can extract `in.nextLine()` into a variable by:

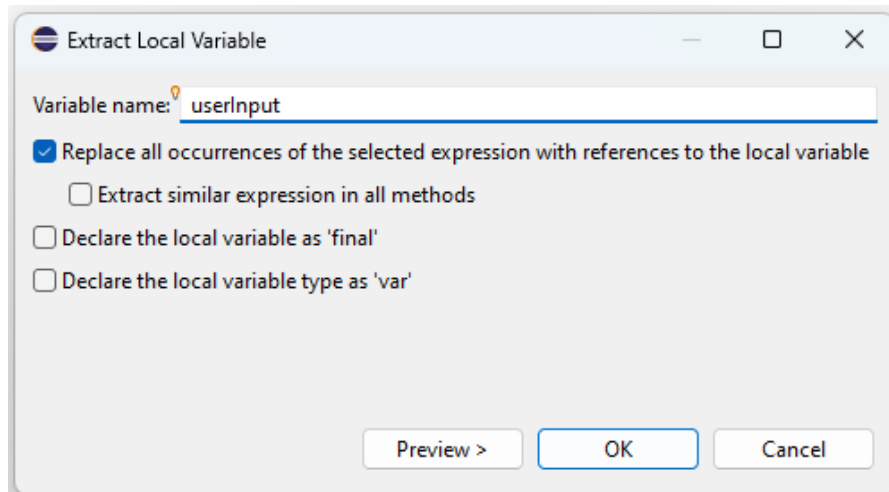
1. Selecting `in.nextLine()`.



2. Right-click and then select **Refactor** → **Extract Local Variable** from the pop-up menu

Move...	Alt+Shift+V
Change Method Signature...	Alt+Shift+C
Extract Method...	Alt+Shift+M
Extract Local Variable...	Alt+Shift+L

3. Enter a new local variable name. Press **OK** to replace all occurrences with the new name, or press **Preview** to review the changes.



The code is updated as shown below:

```
public static void main(String[] args) {
    while (true) {
        Scanner in = new Scanner(System.in);
        System.out.println("Enter RPN expression or \"quit\".");
        // Local Variable extracted
        String userInput = in.nextLine();
        if (userInput.equals("quit")) {
            break;
        } else {
            RPN calc = new RPN();
            System.out.printf("Answer is %f\n", calc.value);
        }
    }
}
```

```
}
```

There are several benefits to introducing variables this way. Firstly, it makes explicit what the code is doing by providing meaningful names to the expressions. Secondly, it makes it easier to debug the code, because we can easily inspect the values that the expressions return. Finally, it can be more efficient in cases where multiple instances of an expression can be replaced with a single variable.

The **Extract Constant** refactoring is similar to **Extract Local Variable**, but you must select a static, constant expression, which the refactoring will convert to a static final constant. This is useful for removing hard-coded numbers and strings from your code.

- **Be as brief as possible:** If you can do something with fewer lines of code without detracting too much from the readability of your code, do so. Using Java libraries' built-in functions or the latest code syntax can often make you write fewer lines of code. Consider the following example of scanning through a file:

```
/**
 * Scans through a file named "someFile.txt" using a Scanner object.
 */
public void traditionalFileScan() {
    Scanner scanner = null;
    try {
        scanner = new Scanner(new File("someFile.txt"));
        while (scanner.hasNext()) {
            // do something
        }
    } catch (FileNotFoundException e) {
        e.printStackTrace();
    } finally {
        if (scanner != null) {
            scanner.close();
        }
    }
}
```

```
}  
  
}
```

This is done via the traditional try-catch-finally block, which is a bit lengthy. The same can be achieved by using newer try-with-resource blocks. For example:

```
public void newFileScan() {  
    try (Scanner scanner = new Scanner(new File("someFile.txt"))) {  
        while (scanner.hasNext()) {  
            // do something  
        }  
    } catch (FileNotFoundException e) {  
        e.printStackTrace();  
    }  
}
```

The new code is more readable!

- **Make sure inputs to functions and output from functions are clear:** Make sure that your functions return real, meaningful output. Make input into your functions as clear as possible.
- **Remove unnecessary code:** As we code, we often create a lot of code that we can, and should, remove from the final version of our app.

So be sure to **delete**:

- Variables that are declared but never used.
- Functions that are never called.
- Code that you commented out because you thought you might need it, but you don't.
- Statements used for debugging. Your code is likely to include things like `System.out` statements that you used to debug your system. For the final version of your app, remember to get rid of these types of statements.

So far, we have considered the general principles for refactoring code. When creating object-oriented programs, there are some more specific guidelines for refactoring to consider. These are discussed next.

Refactoring classes

- **Ensure classes have high levels of cohesion.** Cohesion means that all the methods and properties in a class are closely related to each other and only perform one function. If a class takes ownership of an assortment of unrelated responsibilities, it should be broken up into multiple classes. Each class should only have one responsibility. All methods and data in that class should be strongly related to that responsibility.

For example, consider the following code:

```
class Employee {  
    private String name;  
    private int employeeId;  
    private double salary;  
    public void calculateTax() {  
        // tax calculation  
    }  
    public void requestAnnualLeave() {  
        // request annual leave  
    }  
    public void generatePayroll() {  
        // generate payroll  
    }  
    public void sendEmail() {  
        // send email  
    }  
}
```

This `Employee` class shows a low level of cohesion because it provides a number of unrelated functions that handle different responsibilities. A better approach is to extract these responsibilities into separate classes, each containing only the functions that are directly related to its specific responsibilities. This way, each class will have high cohesion. For instance:

```
class TaxCalculator {  
    public void calculateTax(Employee employee) {  
        // tax calculation  
    }  
}  
  
class AnnualLeaveService {  
    public void requestAnnualLeave(Employee employee) {  
        // request annual leave  
    }  
}  
  
class PayrollGenerator {  
    public void generatePayroll(Employee employee) {  
        // generate payroll  
    }  
}
```

- **Ensure programs with low coupling.** Coupling describes how dependent classes are on each other. Low coupling means that objects are not dependent on each other, so you can change one class without having to make changes to different classes. To enable low coupling, we must make sure that any variables/properties that should belong to an object are declared locally in that class. These properties can also be made private. Then other classes can't get to that data directly. If another class wants to change the properties of a class, they have to use the class's accessor or mutator methods to do this.

Another way to introduce low coupling is to use interfaces or abstract classes instead of concrete classes directly. For instance, consider the following code:

```

/**
 * Calculates a discount based on a fixed percentage and uses it to
 * compute the total price of an order.
 */
class DiscountCalculator {
    private static final double DISCOUNT_PERCENTAGE = 0.1d;

    public static double calculateDiscount(double unitPrice, int
quantity) {
        return unitPrice * quantity * DISCOUNT_PERCENTAGE;
    }
}

class Order {
    public double calculateTotalPrice(double unitPrice, int quantity) {
        double discount =
DiscountCalculator.calculateDiscount(unitPrice, quantity); // Highly
coupled with DiscountCalculator

        return unitPrice * quantity - discount;
    }
}

```

The `Order` class is directly calling another class `DiscountCalculator`'s `calculateDiscount` method. This creates a **high** coupling. Any change to the `DiscountCalculator` class implementation may have unintended consequences in the `Order` class.

A better approach to ensure lower coupling is to define an interface and inject it into the `Order` class to use. For example:

```

/**
 * After refactoring
 */

public interface IDiscountCalculator {

    public double calculateDiscount(double unitPrice, int quantity);

}

public class Order {

    private IDiscountCalculator discountCalculator;

    public Order(IDiscountCalculator discountCalculator) {

        this.discountCalculator = discountCalculator;

    }

    public double calculateTotalPrice(double unitPrice, int quantity) {

        double discount =
discountCalculator.calculateDiscount(unitPrice, quantity); // Use
interface to reduce coupling

        return unitPrice * quantity - discount;

    }

}

```

- **Remove set() methods for fields that cannot be changed:** If a field is supposed to be set at object creation time and not changed afterwards, initialise that field in the object's constructor rather than providing a misleading set() method.
- **Hide methods that are not intended to be used outside the class:** Do this by making these helper methods private.
- **Hide data members:** Public data members are always a bad idea, as they blur the line between interface and implementation. Additionally, they inherently violate encapsulation and limit future flexibility. Consider hiding public data members behind access methods.

Other refactoring

- **Make sure that a loop isn't too long or too deeply nested.** The code inside a loop can be converted into methods. This helps to factor the code better and reduce the loop's complexity. For example, consider the following code:

```
/**
 * Returns the first person who speaks the specified target language.
 */
public Person getPersonWithSpeakingLanguage(String targetLanguage) {
    for (Person person : persons) {
        for (String speakingLanguage : person.getSpeakingLanguages()) {
            if (speakingLanguage.equals(targetLanguage)) {
                return person;
            }
        }
    }
    return null;
}
```

It has double for-loops to find out the first `Person` with a matching speaking language. This can be refactored to be clearer. One way is to extract the inner for-loop into a separate method, for instance:

```
/**
 * After refactoring
 */
private boolean isPersonWithSpeakingLanguage(Person person, String
targetLanguage) {
    for (String speakingLanguage : person.getSpeakingLanguages()) {
        if (speakingLanguage.equals(targetLanguage)) {
```

```

        return true;
    }
}
return false;
}

```

And call this separate method like this:

```

public Person getPersonWithSpeakingLanguage(String targetLanguage) {
    for (Person person : persons) {
        if (isPersonWithSpeakingLanguage(person, targetLanguage)) {
            return person;
        }
    }
    return null;
}

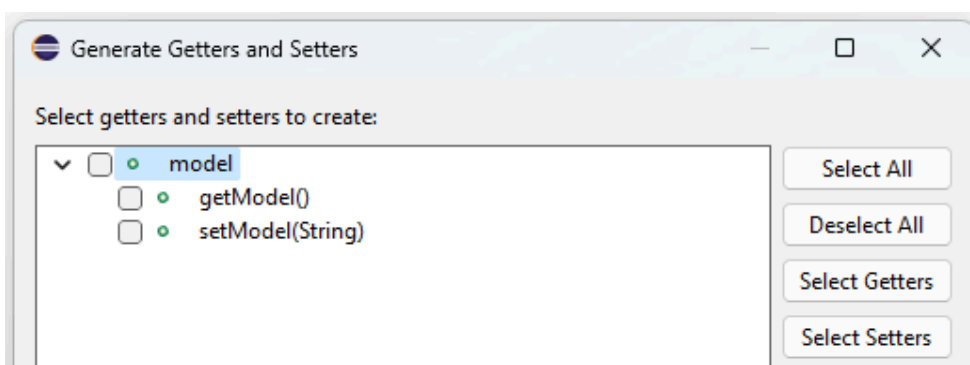
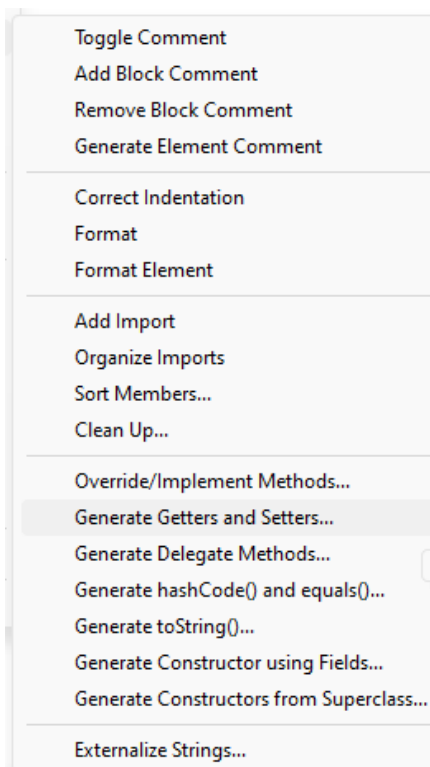
```

The resultant code is a lot easier to understand.

- **Use break or return instead of a loop control variable:** If you have a variable within a loop (like a boolean variable called “done”) that’s used to control the loop exit, use break or return to exit the loop instead.
- **Return as soon as you know the answer instead of assigning a return value within nested if-then-else statements:** Code is often easiest to read and least error-prone if you exit a method as soon as you know the return value. The alternative to setting a return value and then unwinding your way through a lot of logic can be harder to follow.
- **Make sure that a method’s parameter list doesn’t have too many parameters:** Well-factored programs have many small, well-defined methods that don’t need large parameter lists. A long parameter list is a warning that the abstraction of the method interface has not been well thought out. A rule of thumb is not to have more than three parameters.

- Comments are important, but **do not use comments as a crutch to explain bad code.**
- **Encapsulate class variables as private or protected:** Declare your class variables as private or protected. It's not good practice to expose the internal structure of your objects by providing public access to class variables. In order to access private or protected class variables from outside, you should create public setter and getter methods for them. These methods can be generated easily in Eclipse IDE.

To generate these methods, right-click and select **Source → Generate Getters and Setters**. This will display a dialogue box with the proposed getter and setter methods for each field that doesn't already have one.



However, this is not a refactoring because it does not update references to the fields to use the new methods and you'll need to do that yourself if necessary.

This is a great time saver, but it's best used when creating a class initially, or when adding new fields to a class because no other code references these fields yet, and there's, therefore, no other code to change.

- Replace numeric or string literals with **meaningfully named constants**. For instance, replace the literal 3.14 with PI like this:

```
public static final double PI = 3.14d;

public static double calculateCircleArea(double radius) {
    return Math.pow(radius, 2) * PI;
}
```

- Create and use null objects instead of testing for null values. Sometimes a null object will have generic behaviour or data associated with it, such as referring to a resident whose name is not known as “occupant.” In this case, consider moving the responsibility for handling null values out of the client code and into the class — that is, have the `Customer` class define the unknown resident as “occupant” instead of having the `Customer`’s client code repeatedly test for whether the customer’s name is known and substitute “occupant” if not.



Take note

Best practice guidelines

There are many ways of getting a program to work — sort of, some of the time, fingers crossed! Often, a key factor that separates reckless coders from professional software developers is the application of best practice when coding. A good resource when it comes to Java best practice is found [here](#). One of the key goals of refactoring code is to bring your code more in line with best practice guidelines..

Practical task 1

Follow these steps:

- For this task, you are required to refactor the badly written program in the **Factory** folder.
- The factory method pattern is a method that returns one of several possible classes that share a common superclass. It is particularly useful when you don't know beforehand what class you need and the decision can only be made at run time. The superclass that the factory method pattern uses is usually implemented as an interface. An interface is a special type of class that acts as a blueprint for classes to come. It is an abstract type that specifies the behaviour of future classes. An interface will contain method headings, but the methods in an interface don't contain any logic. The logic for the methods specified in the interface is written in the classes that implement the interface. In essence, an interface shows a class what to do, and not how to do it.
- It is not vital for you to be familiar with design patterns for this task, but you can read more about them [here](#).
- For this task, be sure to:
 - Troubleshoot and debug the code first so that it runs correctly.
 - Fix the indentation and formatting of the code so that it adheres to the guidelines provided [here](#).
 - Make sure that all the names of variables, classes, methods, etc. adhere to the guidelines provided [here](#).
 - Refactor the code to improve its quality and readability by applying best practices (e.g. encapsulation, reducing coupling, increasing cohesion, etc.) where suitable.

Be sure to place files for submission inside your task folder and click "Request review" on your dashboard.

Practical task 2

Follow these steps:

- For this task, you are required to refactor the badly written program in the **Decorator** folder.
- The decorator pattern adds something extra to an object that already exists. It wraps an object in order to add functionality or features to the object. The purpose of the pattern is to make it easy to add or remove functions if needed. This is done by wrapping a single object (not class) to add new behaviour. It has an attribute for that object and has all the same methods as the wrapped object.
- It is not vital for you to be familiar with design patterns for this task, but you can read more about them [here](#).
- For this task, be sure to:
 - Troubleshoot and debug the code first so that it runs correctly.
 - Fix the indentation and formatting of the code so that it adheres to the guidelines provided [here](#).
 - Make sure that all the names of variables, classes, methods, etc. adhere to the guidelines provided [here](#).
 - Refactor the code to improve its quality and readability by applying best practices (e.g. encapsulation, reducing coupling, increasing cohesion, etc.) where suitable.

Be sure to place files for submission inside your task folder and click "Request review" on your dashboard.



Share your thoughts

HyperionDev strives to provide internationally excellent course content that helps you achieve your learning outcomes.

Do you think we've done a good job or do you think the content of this task, or this course as a whole, can be improved?

Share your thoughts anonymously using this [**form**](#).
